

Viaje Aventura

Informe Técnico “Desarrollo de Software”

Asignatura: Programación Orientada a Objetos Seguro

Caso de Estudio: Caso “Viaje Aventura”

Sección: TI30V21

Nombre del académico: Rubén Alejandro Schnettler Lucero

Nombre estudiantes: Benjamín Comanato (Líder de Proyecto)

Sergio Antonio Ruiz Espinoza

Bernardo Fuentes Ibaceta

Sede: INACAP Valparaíso

Fecha de entrega: 05 de octubre del 2026



INDICE

1.	INTRODUCCIÓN	5
1.1	Contexto.....	5
1.2	Objetivos	6
1.3	Alcance	6
1.4	Enfoque de trabajo	6
2.	Levantamiento de requerimientos (criterio 4.1.1).....	7
2.1	Análisis de la problemática	7
2.2	De reglas de negocio a requerimientos.....	7
2.3	Requerimientos funcionales	9
2.4	Requerimientos no funcionales.....	10
2.5	Clasificación, priorización y verificación	12
2.6	Supuestos por vacíos del caso.....	12
3.	Modelado de la solución (criterio 4.1.2)	13
3.1	Diagrama de casos de uso.....	13
3.2	Modelos de procesos de negocio (BPMN).....	15
3.3	Diagramas de clases UML.....	17
3.4	Modelo de datos.....	19
3.5	Arquitectura de la solución	20
3.6	Trazabilidad entre requerimientos y modelos (Indicador 4.1.2.I.8)	20
4.	Planificación con metodologías ágiles (criterio 4.1.3).....	23
4.1	Marco de trabajo y roles	23
4.2	Product Backlog priorizado.....	24
4.3	Sprint Backlog por iteración.....	26
4.4	Tiempos, entregables y riesgos	27
5.	Implementación en Python con persistencia (criterio 4.1.4)	28
5.1	Tecnologías y organización del código	28
5.2	Programación orientada a objetos aplicada	28
5.3	Persistencia de datos	29
5.4	Funcionalidades CRUD	29
5.5	Pruebas y verificación	30
5.6	Limitaciones conocidas	31
6.	Acceso seguro a la solución (criterio 4.1.5)	31

6.1 Librerías de repositorios oficiales y criterios de calidad.....	31
6.2 Mecanismo de autenticación	32
6.3 Validación rigurosa de credenciales (Indicador 4.1.5.G.18)	32
6.4 Protección de la información sensible (Indicador 4.1.5.I.19).....	33
6.5 Análisis de amenazas y controles.....	33
6.6 Evaluación y optimización de la seguridad con apoyo de IA (Indicador 4.1.5.I.20).....	34
6.7 Riesgos residuales y recomendaciones.....	35
7. Uso crítico de herramientas de inteligencia artificial.....	35
7.1 Proceso de revisión aplicado.....	35
7.2 Registro de contribuciones, análisis y decisiones	35
7.3 Riesgos del uso de IA y cómo se controlaron	37
8. Conclusiones.....	37
Referencias.....	38

1. INTRODUCCIÓN

1.1 Contexto

Viajes Aventura es una agencia de Valparaíso creada por tres socios que arma y vende paquetes turísticos combinando destinos que ellos mismos han recorrido. Hoy opera sin sistema: el catálogo vive en una planilla compartida, las reservas en un cuaderno y los datos de los clientes (nombre, RUT, teléfono y correo) en el cuaderno, en la planilla y en las conversaciones de mensajería de cada socio. En la última temporada registraron 214 reservas, de las cuales 19 estaban duplicadas, 6 superaron el cupo del paquete, 3 se tomaron sobre paquetes con fecha de salida vencida y 4 paquetes se publicaron con un precio distinto del cobrado.

Este informe documenta la solución completa: el levantamiento de requerimientos, el modelado con UML y BPMN, la planificación ágil, la implementación en Python con persistencia de datos y el acceso seguro, junto con el uso crítico de herramientas de inteligencia artificial durante el desarrollo.

1.2 Objetivos

Objetivo general. Desarrollar un sistema en Python que permita a Viajes Aventura gestionar destinos, paquetes turísticos y reservas de clientes, garantizando la integridad de la información y la protección de las credenciales y de los datos sensibles.

- Identificar, clasificar, priorizar y validar los requerimientos funcionales y no funcionales a partir del caso (criterio 4.1.1).
- Modelar estructura y procesos con casos de uso, BPMN y diagramas de clases UML, asegurando trazabilidad con los requerimientos (4.1.2).
- Planificar el trabajo con Scrum: roles, Product Backlog, Sprint Backlog, tiempos y entregables (4.1.3).
- Implementar la solución aplicando programación orientada a objetos, persistencia en base de datos y CRUD, evaluando críticamente el código sugerido por IA (4.1.4).
- Implementar autenticación y protección de datos con librerías de repositorios oficiales y evaluar la seguridad del sistema (4.1.5).

1.3 Alcance

Tabla 1. Alcance de la versión 1: dentro y fuera del alcance.

Dentro del alcance (versión 1)	Fuera del alcance (acordado por los socios)
Destinos: registrar, modificar, dejar no disponible y listar. Paquetes: crear combinando destinos, definir fechas y cupo, calcular el precio y consultar disponibilidad. Reservas: registro y autenticación de clientes, reservar, almacenar y consultar el historial propio. Seguridad: autenticación, validación de todo dato ingresado y protección de credenciales y datos sensibles.	Pasarela de pago y verificación automática de transferencias · facturación electrónica ante el SII · aplicación para teléfonos · integración con aerolíneas, hoteles u operadores · envío de correos o mensajería al cliente · informes de gestión · contabilidad y remuneraciones.

El caso declara explícitamente que contiene vacíos (qué ocurre cuando un cliente desiste, cómo se comporta un paquete al terminar su temporada, quién modifica el catálogo). Cada vacío se resolvió con un supuesto declarado y fundamentado técnicamente en la sección 2.6.

1.4 Enfoque de trabajo

El equipo trabajó con Scrum en cuatro sprints semanales (semanas 15 a 18 de la asignatura). La solución se construyó en capas (presentación, servicios, dominio, acceso a datos y seguridad) sobre Python 3.12 y SQLite, con las librerías argon2-cffi y cryptography publicadas en PyPI. Cada regla de negocio del caso se tradujo a un requerimiento, se modeló, se implementó y se verificó con al menos una prueba automatizada; la trazabilidad completa se presenta en la sección 3.6.

2. Levantamiento de requerimientos (criterio 4.1.1)

2.1 Análisis de la problemática

Las cifras y testimonios del caso se analizaron para separar el síntoma observado de su causa raíz. Esto evita proponer requerimientos que solo “parchen” el síntoma (por ejemplo, agregar una columna a la planilla) y orienta al sistema a eliminar la causa.

Tabla 2. Evidencia del caso, causa raíz y respuesta del sistema.

Evidencia del caso	Causa raíz identificada	Respuesta del sistema (req.)
19 reservas duplicadas (misma reserva escrita por dos socios)	No existe un registro único ni identificación del cliente; cada socio anota por su cuenta	Reserva ligada a un cliente autenticado; una sola reserva activa por cliente y paquete (RF-11, RF-18)
6 reservas por sobre el cupo	El cupo “se lleva en la cabeza”; no hay un dato de cupo disponible que el sistema compare	Cupo disponible calculado = cupo máximo – personas reservadas, verificado en transacción (RF-11, RNF-05)
3 reservas sobre fechas vencidas	Nadie compara contra la fecha del día; los paquetes vencidos siguen visibles	Rechazo de reservas con fecha de salida pasada; el cliente solo ve paquetes vigentes (RF-08, RF-11)
4 paquetes con precio distinto del cobrado	El costo de un destino cambia después de vendido el paquete y la planilla recalcula	Precio congelado al publicar; total de la reserva fijo (RF-07, RF-11)
Error de un cero al armar un paquete (publicado a la décima parte)	Cálculo manual: copiar costos, sumar y aplicar margen a mano	Cálculo automático del precio = suma de costos + margen, con margen validado (RF-06)
4 destinos sin operar aún visibles; no se borran porque están en paquetes vendidos	No hay baja lógica; borrar rompería el contenido de paquetes ya vendidos	Eliminar si está libre; si está en algún paquete, marcar NO DISPONIBLE (RF-03, RF-04)
31 consultas respondidas revisando el cuaderno hoja por hoja; no se sabe cuántas personas han viajado	Datos sin estructura ni consulta	Base de datos persistente e historial de reservas por cliente (RF-12)
RUT y teléfono en una planilla enviada por correo; riesgo de pérdida	Datos sensibles en texto plano, sin control de acceso	Cifrado en reposo, contraseñas con hash, acceso por rol y ocultamiento en listados (RF-13, RF-15, RNF-01, RNF-02)
Destinos repetidos con nombres distintos; columnas agregadas por cada socio	Sin validación ni restricciones de integridad	Nombre de destino único y validación de todo dato de entrada (RF-01, RF-14)

2.2 De reglas de negocio a requerimientos

El caso entrega 17 reglas de negocio (R1 a R17) y advierte que una regla describe cómo funciona el negocio, mientras que un requerimiento describe qué debe hacer el sistema. Por eso cada regla se tradujo a uno o más requerimientos verificables, redactados con la estructura “El sistema deberá...” y acompañados de un criterio de aceptación medible. Ninguna regla quedó sin cobertura (véase la matriz de cobertura de la sección 2.5).

2.3 Requerimientos funcionales

Prioridad según MoSCoW: M = Debe (imprescindible), S = Debería (aporta valor alto, no bloquea), C = Podría. Origen: regla de negocio (R#), entrevista o supuesto (S#).

Tabla 3. Requerimientos funcionales (RF).

ID	Requerimiento	Prior.	Origen	Criterio de aceptación
RF-01	El sistema deberá permitir al administrador registrar un destino con nombre, zona, descripción, duración en días y costo base por persona.	M	R1, R2	Se rechaza un nombre repetido (sin distinguir mayúsculas) y un costo ≤ 0 .
RF-02	El sistema deberá permitir al administrador modificar los datos de un destino existente.	M	R1, R2, R7	Los datos se actualizan y los paquetes ya publicados conservan su precio.
RF-03	El sistema deberá eliminar un destino que no pertenece a ningún paquete y, si pertenece a alguno, marcarlo como no disponible sin eliminarlo.	M	R8	Destino libre desaparece; destino en paquete queda con disponible = No y el paquete vendido conserva su contenido.
RF-04	El sistema deberá listar los destinos y ofrecer para paquetes nuevos solo los disponibles.	M	R8	El listado muestra el estado; un destino no disponible no puede agregarse a un paquete.
RF-05	El sistema deberá permitir crear un paquete con nombre, fecha de salida, fecha de regreso, cupo máximo y entre 2 y 5 destinos distintos.	M	R3, R4, R5	Se rechazan < 2 o > 5 destinos, destinos repetidos, $\text{regreso} \leq \text{salida}$ y $\text{cupo} \leq 0$; un destino puede estar en varios paquetes.
RF-06	El sistema deberá calcular el precio por persona como la suma de los costos base de los destinos más el margen de operación (20 % por defecto, nunca negativo).	M	R6	Destinos $120.000 + 310.000$ con margen 20 % $\rightarrow 516.000$; un margen negativo se rechaza.
RF-07	El sistema deberá fijar el precio al publicar el paquete, sin alterarlo si el costo de un destino cambia después.	M	R7	Tras modificar el costo de un destino, el paquete publicado mantiene su precio.
RF-08	El sistema deberá permitir consultar los paquetes vigentes con su precio, destinos y cupo disponible.	M	R14, R15	El cliente no ve paquetes con salida vencida; el cupo mostrado = máximo – reservado.

ID	Requerimiento	Prior.	Origen	Criterio de aceptación
RF-09	El sistema deberá permitir que un cliente se registre con nombre, RUT, correo, teléfono y contraseña.	M	R9, R10	Correo único; RUT válido (módulo 11); contraseña que cumple la política.
RF-10	El sistema deberá autenticar a los usuarios por correo y contraseña y abrir una sesión.	M	R11	Credenciales correctas abren sesión; incorrectas se rechazan con mensaje genérico.
RF-11	El sistema deberá permitir a un cliente autenticado reservar un paquete indicando la cantidad de personas, y registrar la fecha de emisión y el total.	M	R12–R16	Cantidad ≥ 1 ; se rechaza sobre el cupo disponible o con salida pasada; total = precio $\times$ personas, inmodificable.
RF-12	El sistema deberá permitir al cliente consultar el historial de sus propias reservas.	M	R11	Un cliente solo ve sus reservas; nunca las de otro.
RF-13	El sistema deberá restringir las operaciones según el rol: solo administradores gestionan el catálogo y solo clientes autenticados reservan.	M	R11, S1	Un cliente que intenta gestionar destinos, o un token inválido, reciben un rechazo de autorización.
RF-14	El sistema deberá validar todo dato ingresado (tipo, formato, rango y longitud) antes de procesarlo.	M	Alcance	Entradas inválidas o con código SQL se rechazan sin alterar la base de datos.
RF-15	El sistema no deberá mostrar el RUT ni el teléfono completo en listados ni en mensajes de error.	M	R17	Los datos aparecen enmascarados y los mensajes de error no los incluyen.
RF-16	El sistema deberá permitir al cliente cancelar una reserva propia de un paquete aún no iniciado, liberando el cupo.	S	S4	La reserva pasa a CANCELADA y el cupo disponible aumenta; su total no se modifica.
RF-17	El sistema deberá permitir al administrador listar todos los paquetes, incluidos los vencidos.	S	S3, entrevista a M. Bórquez	El administrador ve el estado Vigente / Vencido de cada paquete.
RF-18	El sistema deberá impedir que un mismo cliente mantenga dos reservas activas en el mismo paquete.	M	S6, 19 duplicadas	El segundo intento se rechaza; tras cancelar, puede volver a reservar.

2.4 Requerimientos no funcionales

Los requerimientos no funcionales (RNF) describen cómo debe comportarse el sistema en seguridad, integridad, rendimiento, usabilidad, mantenibilidad y portabilidad. Se redactan con la misma estructura que los funcionales, con su categoría, prioridad MoSCoW y un criterio de verificación. El método y la evidencia con que se comprobó cada uno se presentan en la tabla de verificación de los RNF.

Tabla 4. Requerimientos no funcionales (RNF).

ID	Categoría	Requerimiento	Prior.	Criterio de verificación
RNF-01	Seguridad	Las contraseñas deberán almacenarse solo como hash Argon2id con sal; nunca tal como las escribe el cliente (R10).	M	La base de datos no contiene la contraseña y el hash comienza con \$argon2id\$.
RNF-02	Confidencialidad	El RUT y el teléfono deberán almacenarse cifrados en reposo con cifrado autenticado (Fernet) (R17).	M	El campo almacenado no contiene el dato en claro y solo se descifra con la clave.
RNF-03	Seguridad	Toda consulta SQL deberá ser parametrizada para evitar inyección SQL.	M	Las cargas de ataque (por ejemplo, una comilla seguida de OR 1=1) no alteran ni exponen datos.
RNF-04	Seguridad de acceso	El acceso deberá bloquear la cuenta durante 15 minutos tras 5 intentos fallidos, usar mensajes genéricos y expirar la sesión tras 30 minutos de inactividad.	M	La cuenta se bloquea y se libera por tiempo; el mensaje es el mismo para correo inexistente y clave incorrecta.
RNF-05	Integridad	Las reglas críticas deberán reforzarse en la base de datos (CHECK, UNIQUE, FOREIGN KEY) y las reservas deberán ejecutarse en una transacción para que el cupo sea consistente.	M	El esquema contiene las restricciones y la reserva verifica el cupo y guarda de forma atómica.
RNF-06	Rendimiento	Las operaciones de catálogo y de reserva deberán responder en menos de 1 segundo con el volumen de una temporada (aproximadamente 214 reservas).	S	Medición con la base de demostración (tiempos en la tabla de verificación de RNF).
RNF-07	Usabilidad	La interfaz deberá usar menús numerados y mensajes claros en español, sin mostrar trazas técnicas al usuario.	S	Los errores inesperados se registran en el log y al usuario se le informa con un mensaje genérico.
RNF-08	Mantenibilidad	El código deberá organizarse en capas, con programación orientada a objetos y pruebas automatizadas de las reglas de negocio.	M	27 pruebas automatizadas aprobadas y módulos de responsabilidad única.
RNF-09	Portabilidad	El sistema deberá ejecutarse en Windows, Linux o macOS con Python 3.10 o superior y sin instalar un motor de base de datos.	M	Base de datos SQLite en un archivo y dependencias declaradas en requirements.txt.

ID	Categoría	Requerimiento	Prior.	Criterio de verificación
RNF-10	Trazabilidad	El sistema deberá registrar eventos de seguridad (registros e inicios de sesión fallidos) en un log sin incluir datos sensibles.	S	El archivo app.log no contiene RUT, teléfono ni contraseñas.
RNF-11	Exactitud	Los montos deberán manejarse como enteros (CLP) y el margen con aritmética Decimal, evitando errores de coma flotante.	M	Precios y totales exactos en las pruebas de cálculo.

2.5 Clasificación, priorización y verificación

Clasificación. Los requerimientos se organizaron en tres ejes: gestión de catálogo (RF-01 a RF-08 y RF-17), reservas y clientes (RF-09 a RF-12, RF-16 y RF-18) y seguridad transversal (RF-13 a RF-15 y RNF-01 a RNF-04 y RNF-10). Los requerimientos de calidad restantes (RNF-05 a RNF-09 y RNF-11) se asocian al conjunto de la solución.

Criterios de priorización. Se aplicó MoSCoW con tres criterios: (1) valor, si elimina una de las fallas medidas en la temporada; (2) riesgo, si compromete datos sensibles o dinero; y (3) dependencia técnica, si otros requerimientos no pueden construirse sin él (por ejemplo, la autenticación RF-10 precede a la reserva RF-11).

Tabla 5. Priorización MoSCoW de los requerimientos.

Prioridad	Requerimientos	Justificación
Debe (M)	RF-01 a RF-15; RNF-01 a RNF-05, RNF-08, RNF-09 y RNF-11	Resuelven directamente las fallas medidas en la temporada (sobrecupo, fechas vencidas, precios distintos, duplicados) y protegen los datos sensibles; sin ellos el sistema no cumple el alcance.
Debería (S)	RF-16, RF-17 y RF-18; RNF-06, RNF-07 y RNF-10	Mejoran la operación y cubren los vacíos del caso (desistimiento, paquetes vencidos), pero la primera versión puede operar sin ellos.

Verificación frente a la problemática (Indicador 4.1.1.I.4). Cada evidencia de la sección 2.1 tiene al menos un requerimiento que elimina su causa raíz, y cada regla de negocio R1 a R17 está cubierta por un requerimiento con criterio de aceptación:

Tabla 6. Cobertura de las reglas de negocio R1 a R17 por requerimiento.

Regla	Cubierta por	Regla	Cubierta por	Regla	Cubierta por
R1	RF-01	R7	RF-07	R13	RF-11
R2	RF-01, RF-02	R8	RF-03, RF-04	R14	RF-08, RF-11
R3	RF-05	R9	RF-09	R15	RF-08, RF-11
R4	RF-05	R10	RF-09, RNF-01	R16	RF-11
R5	RF-05	R11	RF-10, RF-12, RF-13	R17	RF-15, RNF-02
R6	RF-06	R12	RF-11	—	—

Lo declarado fuera de alcance (pago automático, facturación, correos, informes) no genera requerimientos. Las expectativas de los tres socios entrevistados quedan cubiertas: Paulina (cálculo automático del precio y baja lógica de destinos), Matías (control de cupo y de fechas vencidas) e Ignacio (datos protegidos, conteo e historial consultable).

2.6 Supuestos por vacíos del caso

Tabla 7. Supuestos adoptados ante los vacíos del caso.

ID	Vacío detectado	Supuesto adoptado	Fundamento técnico
S1	¿Quién puede modificar el catálogo?	Existe el rol ADMINISTRADOR; el primer administrador se crea por un comando de consola. Los clientes no pueden gestionar destinos ni paquetes.	Principio de mínimo privilegio; evita que un cliente altere precios. Un administrador no se autorregistra para impedir escalamiento de privilegios.
S2	¿Cuándo “se publica” un paquete?	Crear un paquete equivale a publicarlo: el precio se calcula y se congela en ese momento.	Simplifica la versión 1 sin perder la regla R7; un estado borrador puede agregarse después.
S3	¿Qué ocurre con un paquete cuando termina su temporada?	El paquete no se borra ni cambia: queda vencido al pasar su fecha de salida. No admite reservas y el cliente no lo ve; el administrador sí.	Conserva el historial de reservas y evita el error de vender paquetes vencidos (R15). El estado se deriva de la fecha, no se almacena, por lo que no puede quedar desactualizado.
S4	¿Qué pasa cuando un cliente desiste?	El cliente puede cancelar su reserva antes de la salida: pasa a CANCELADA, libera el cupo y conserva su registro. Reembolsos quedan fuera del sistema.	No se borran filas (auditoría); el cupo solo cuenta reservas ACTIVAS. Coherente con “sin pasarela de pago”.
S5	¿Cómo se considera el pago?	El pago por transferencia se verifica manualmente fuera del sistema; la reserva nace ACTIVA.	La verificación automática está fuera de alcance.
S6	¿Puede un cliente reservar dos veces el mismo paquete?	Solo una reserva activa por cliente y paquete; para cambiar personas se cancela y se reserva de nuevo.	Evita duplicados como los 19 detectados; se refuerza con un índice único parcial en la base de datos.
S7	¿Moneda y formato de montos?	Pesos chilenos como enteros; margen con Decimal.	Evita errores de redondeo de coma flotante en precios.
S8	¿Límites del margen?	Margen entre 0 % y 500 %.	La regla solo exige “no negativo”; el tope protege de errores de digitación (como el cero de más reportado).
S9	¿La “temporada” es una entidad?	No: se deriva de las fechas del paquete.	Evita duplicar información y posibles inconsistencias.
S10	¿Tipo de interfaz y despliegue?	Aplicación de consola con base SQLite local.	Tres socios y ~214 reservas por temporada no justifican un servidor; la capa de servicios permite cambiar la interfaz después.

3. Modelado de la solución (criterio 4.1.2)

El modelado representa la solución desde tres perspectivas complementarias: la interacción entre usuarios y sistema (casos de uso), los procesos de negocio (BPMN) y la estructura orientada a objetos y de datos (diagramas de clases y modelo entidad-relación). Los identificadores RF, CU, BPMN y R permiten rastrear cada elemento hasta su requerimiento (sección 3.6).

3.1 Diagrama de casos de uso

Participan dos actores, los mismos roles del caso: Cliente y Administrador. La autenticación (CU-02) está incluida en todas las operaciones protegidas, y el cálculo del precio (CU-09) forma parte de la creación del paquete.

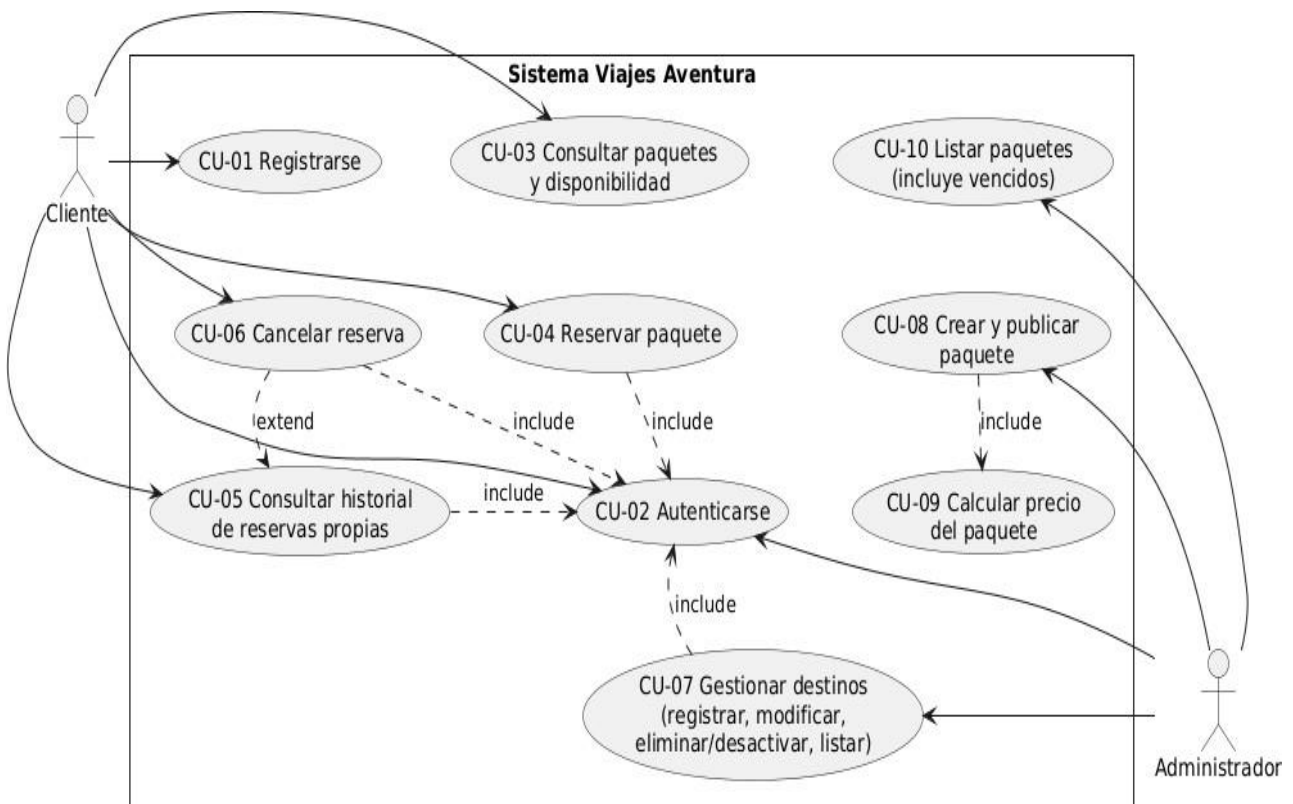


Figura 1. Diagrama de casos de uso de Viajes Aventura (UML).

Tabla 8. Especificación de los casos de uso.

ID	Caso de uso	Actor	Precondición	Flujo principal y resultado	Requerimientos
CU-01	Registrarse	Cliente	Ninguna	Ingresa nombre, RUT, correo, teléfono y contraseña; el sistema valida, cifra datos sensibles, aplica hash a la clave y crea el cliente.	RF-09, RF-14, RNF-01, RNF-02
CU-02	Autenticarse	Cliente / Administrador	Usuario registrado	Ingresa correo y contraseña; el sistema verifica bloqueo y hash, y abre una sesión con rol. Falla con mensaje genérico.	RF-10, RF-13, RNF-04
CU-03	Consultar paquetes y disponibilidad	Cliente	Sesión activa	Lista los paquetes vigentes con destinos, precio por persona y cupo disponible.	RF-08
CU-04	Reservar paquete	Cliente	Sesión activa; paquete vigente con cupo	Elige paquete y cantidad; el sistema valida reglas R14–R16, calcula el total y guarda la reserva de forma atómica.	RF-11, RF-18, RNF-05
CU-05	Consultar historial de reservas	Cliente	Sesión activa	Muestra solo las reservas del cliente autenticado, con RUT y teléfono ocultos.	RF-12, RF-15
CU-06	Cancelar reserva	Cliente	Reserva propia activa de un paquete no iniciado	Cambia la reserva a CANCELADA y libera el cupo.	RF-16
CU-07	Gestionar destinos	Administrador	Sesión de administrador	Registra, modifica, lista y elimina o desactiva destinos según R8.	RF-01 a RF-04, RF-13
CU-08	Crear y publicar paquete	Administrador	Sesión de administrador; ≥ 2 destinos disponibles	Elige 2 a 5 destinos, fechas, cupo y margen; el sistema valida y publica con precio congelado.	RF-05, RF-07
CU-09	Calcular precio del paquete	Sistema (incluido)	Destinos elegidos	Precio = suma de costos base + margen (Decimal).	RF-06, RNF-11
CU-10	Listar paquetes (incluye vencidos)	Administrador	Sesión de administrador	Muestra todos los paquetes con su estado Vigente / Vencido y cupos.	RF-17

3.2 Modelos de procesos de negocio (BPMN)

Se modelaron cinco procesos con carriles para cada participante (usuario y sistema). Los eventos de inicio son círculos verdes; los de fin, círculos gruesos (con una X cuando el proceso termina en rechazo); las compuertas exclusivas (rombos) representan las decisiones que implementan las reglas de negocio.

BPMN 1 · Registro de cliente

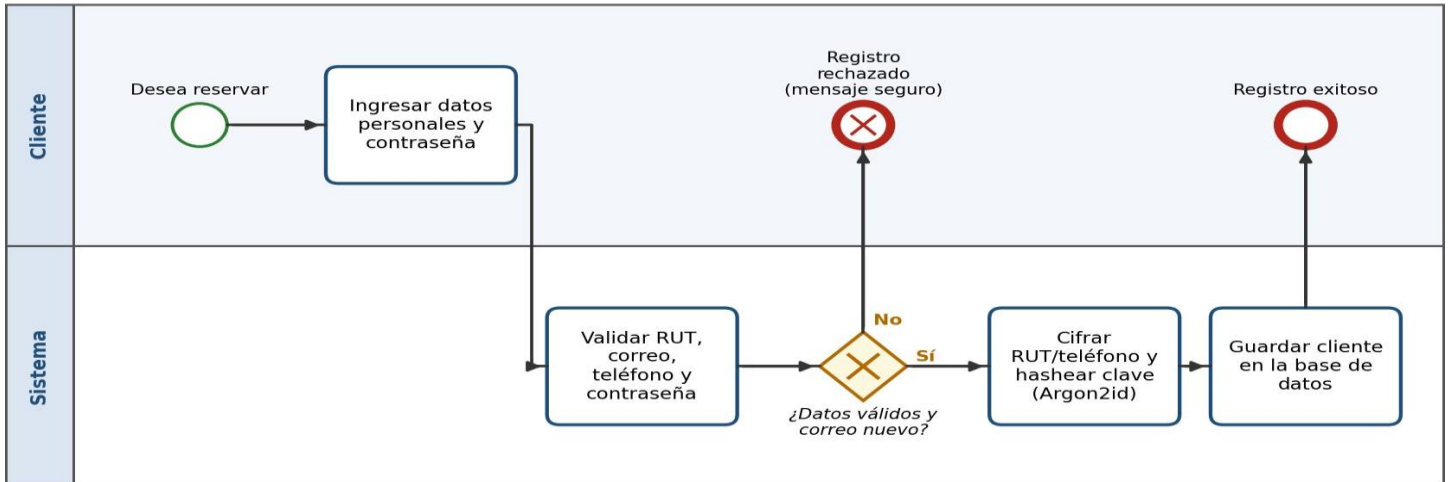


Figura 2. BPMN 1 · Registro de cliente (RF-09, R9, R10, R17).

BPMN 2 · Autenticación de usuario

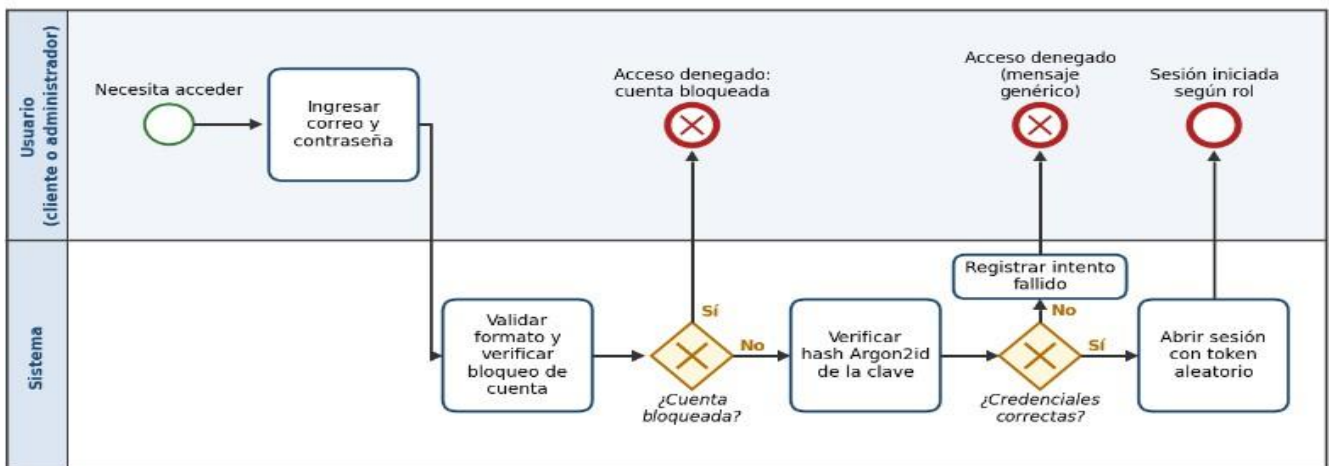


Figura 3. BPMN 2 · Autenticación (RF-10, RNF-04, R11).

BPMN 3 · Gestión de paquetes: crear y publicar

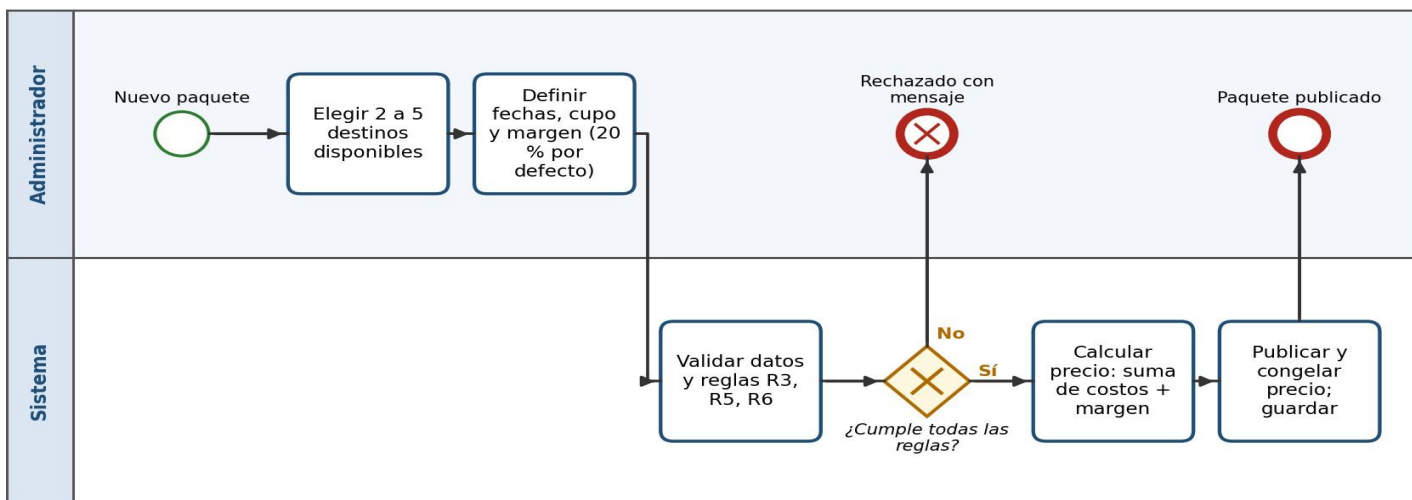


Figura 4. BPMN 3 · Gestión de paquetes: crear y publicar (RF-05, RF-06, RF-07; R3, R5, R6, R7).

BPMN 4 · Gestión de destinos: eliminar o desactivar

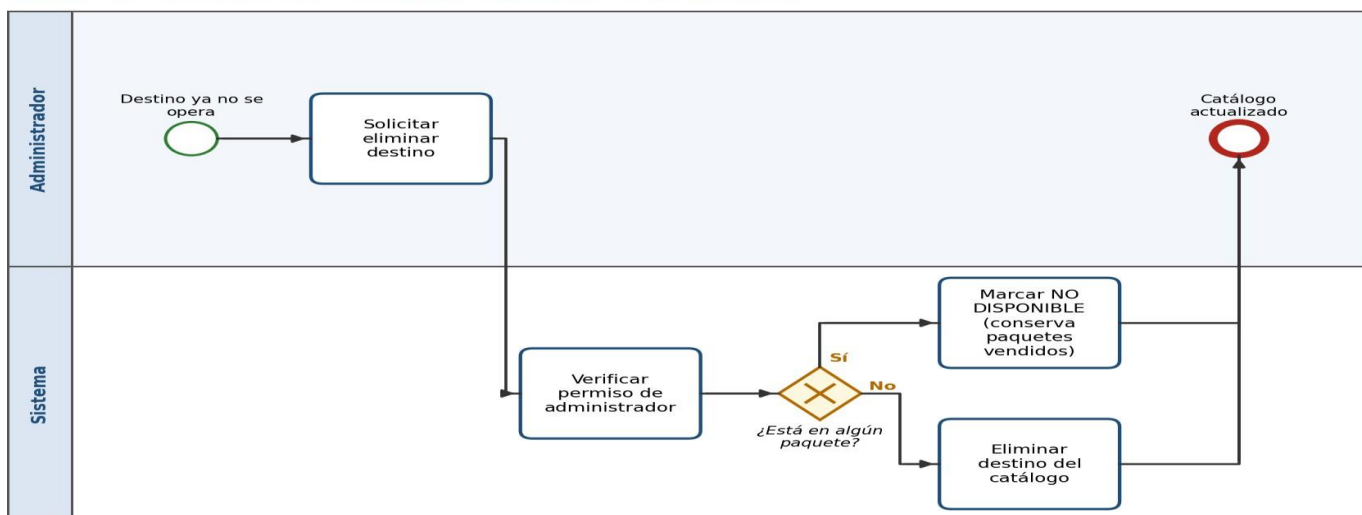


Figura 5. BPMN 4 · Gestión de destinos: eliminar o desactivar (RF-03, R8).

BPMN 5 · Reserva de un paquete turístico

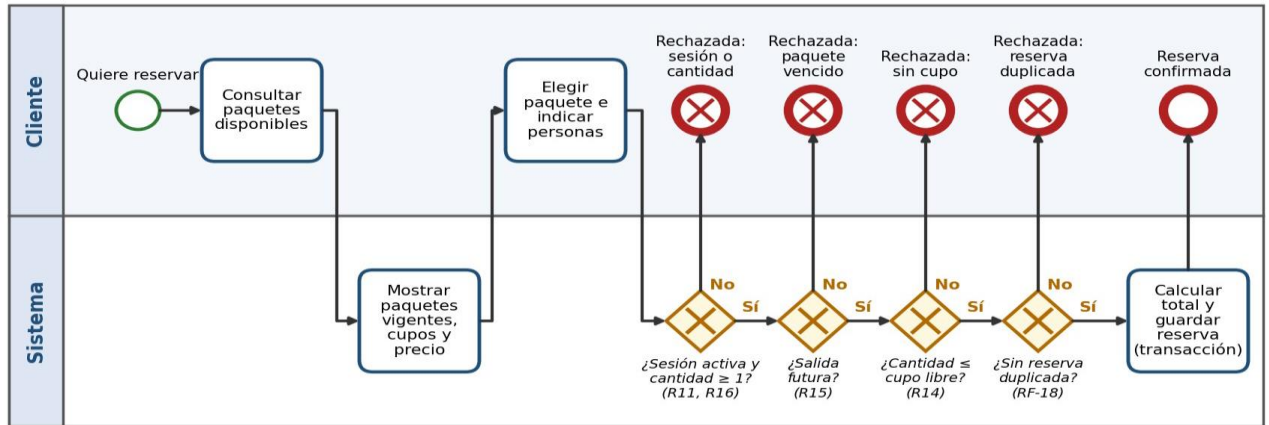


Figura 6. BPMN 5 · Reserva de un paquete (RF-08, RF-11, RF-18; R12 a R16).

Lectura de las decisiones. En el proceso de reserva, cuatro compuertas rechazan la solicitud en orden: (1) sesión activa y cantidad mínima (R11, R16); (2) fecha de salida contra la fecha del día (R15); (3) cantidad solicitada contra el cupo disponible que el sistema calcula (R14); y (4) que el cliente no tenga ya una reserva activa en el mismo paquete (RF-18). Las compuertas 2 y 3 se separan porque, como señala la pista del caso, comparan datos distintos: la fecha del día y un cupo que el sistema debe llevar. Solo si las cuatro respuestas son afirmativas se calcula el total y se guarda la reserva en una transacción (R13). En la baja de destinos, la compuerta decide entre eliminar o desactivar (R8), lo que evita el problema de Paulina de “filas sin nada” en paquetes ya vendidos.

3.3 Diagramas de clases UML

3.3.1 Clases de dominio

El dominio aplica abstracción (Usuario es abstracta), herencia (Cliente y Administrador), polimorfismo (cada rol define sus permisos) y encapsulamiento (atributos privados con validación en los setters). Las multiplicidades reflejan las reglas: un paquete combina de 2 a 5 destinos (R3) y un destino puede estar en varios paquetes (R4); un cliente realiza muchas reservas y cada reserva corresponde a un único paquete (R12).

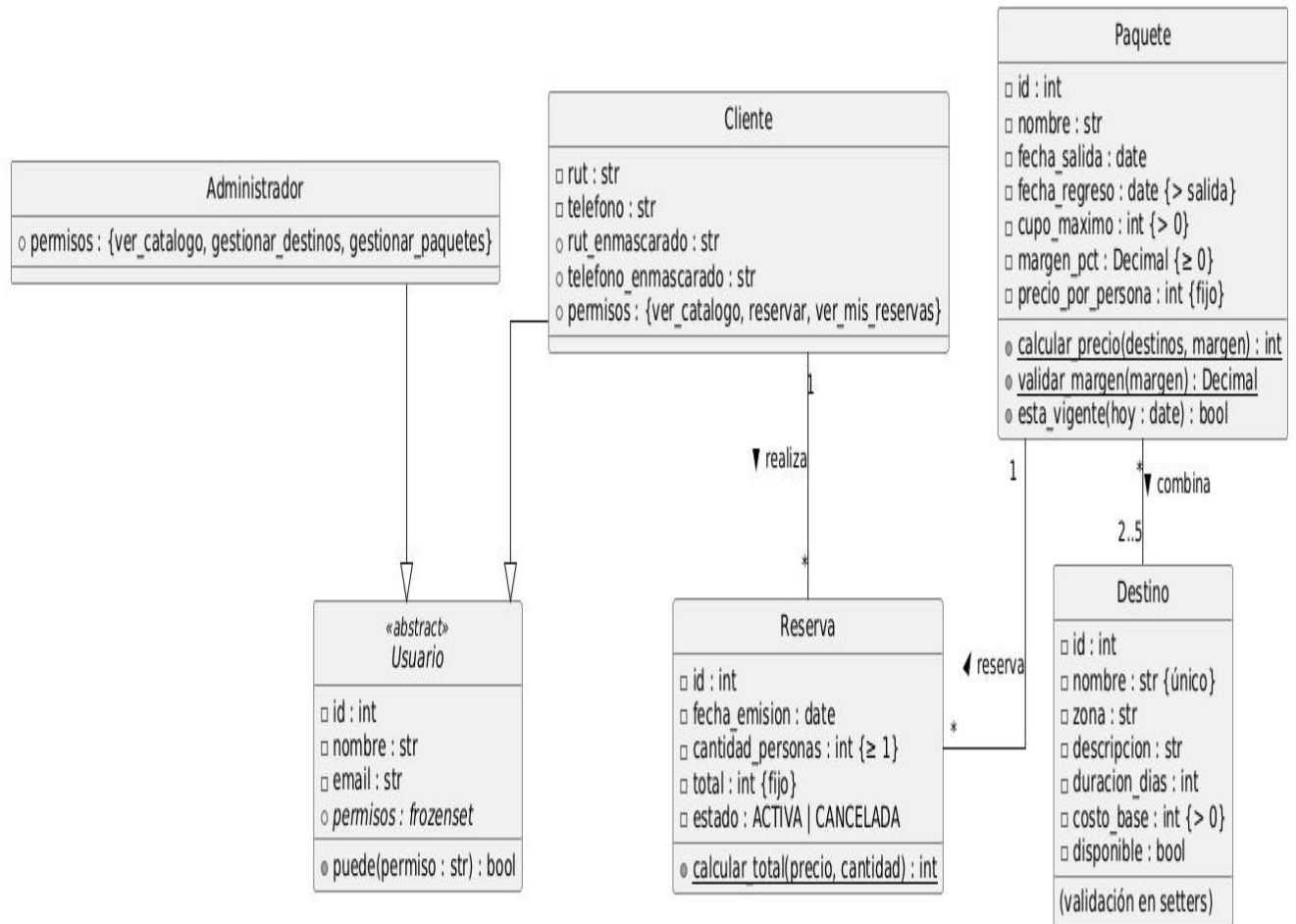


Figura 7. Diagrama de clases de dominio.

3.3.2 Clases de servicios, acceso a datos y seguridad

La lógica de negocio vive en tres servicios que colaboran con repositorios (acceso a datos) y con componentes de seguridad. La interfaz de consola solo depende de los servicios, nunca de la base de datos, lo que permite reemplazarla por una interfaz web sin tocar las reglas.

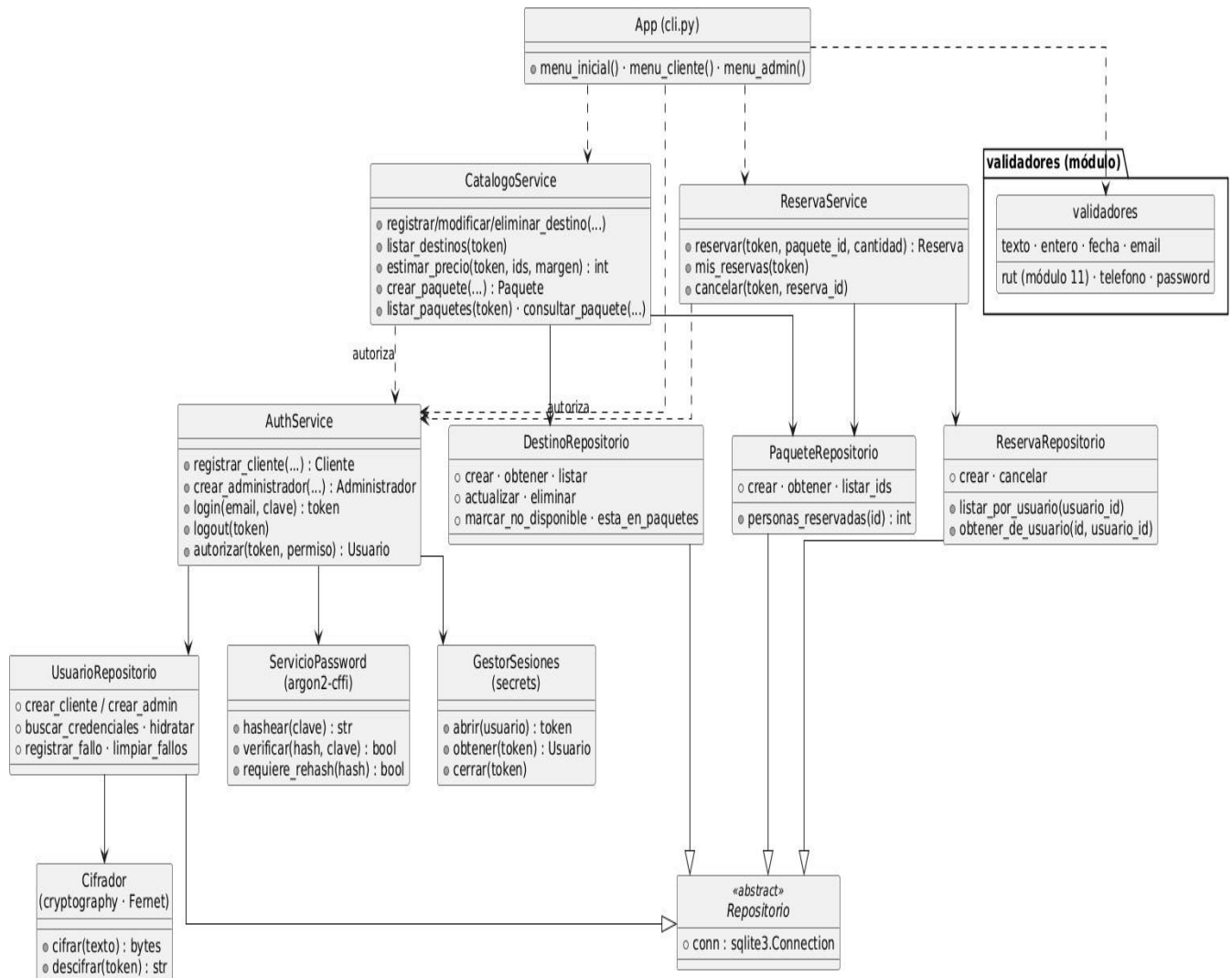


Figura 8. Diagrama de clases de servicios, repositorios y seguridad.

3.4 Modelo de datos

La base de datos SQLite refuerza las reglas críticas con restricciones propias (CHECK, UNIQUE, FOREIGN KEY e índice único parcial), de modo que aunque una capa superior fallara, los datos inválidos no podrían persistirse. La tabla paquete_destinos resuelve la relación muchos a muchos y guarda una instantánea del costo aplicado; su clave foránea hacia destinos usa ON DELETE RESTRICT, que impide borrar un destino usado por un paquete (R8).

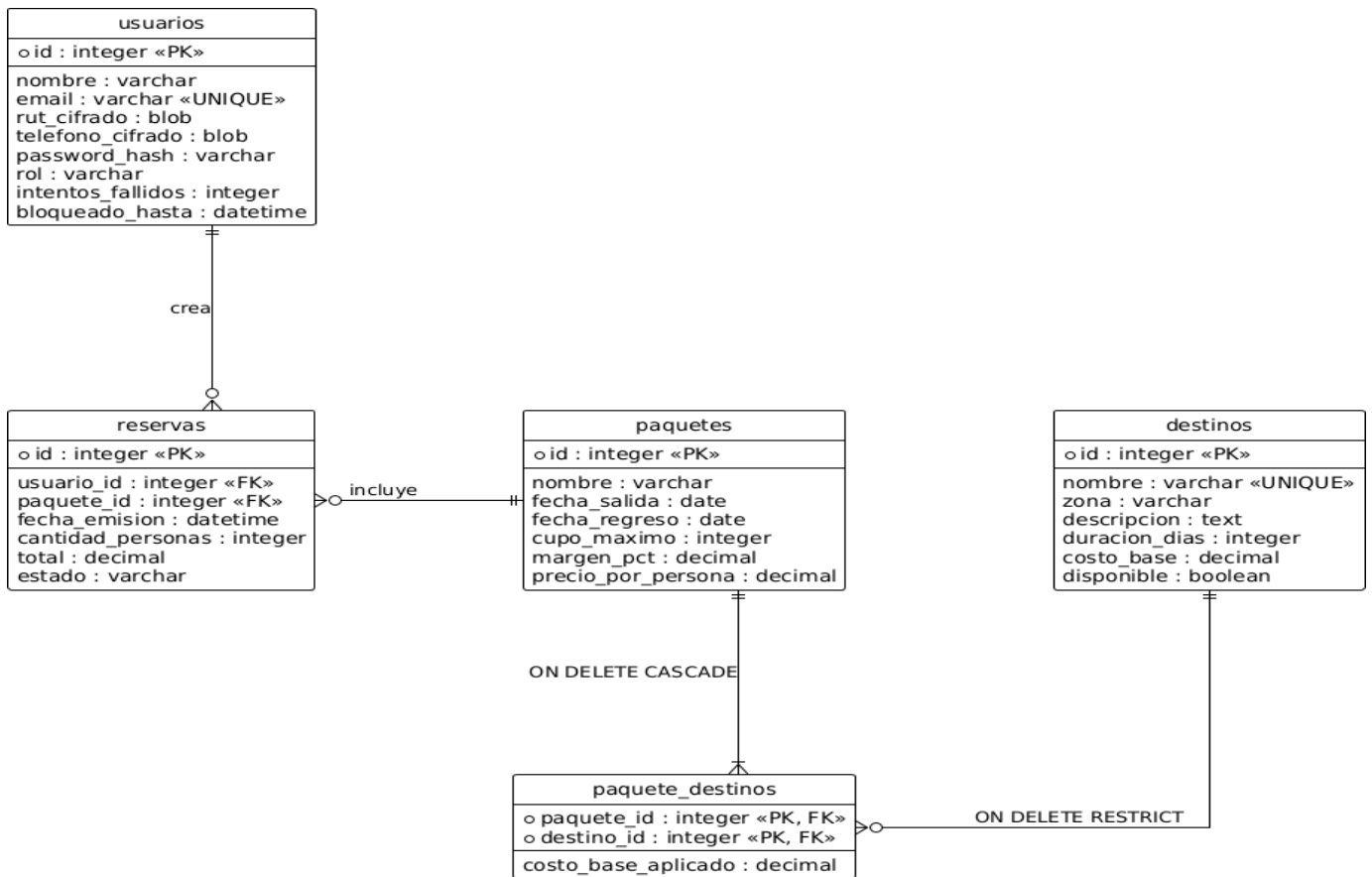


Figura 9. Modelo entidad-relación de la base de datos.

3.5 Arquitectura de la solución

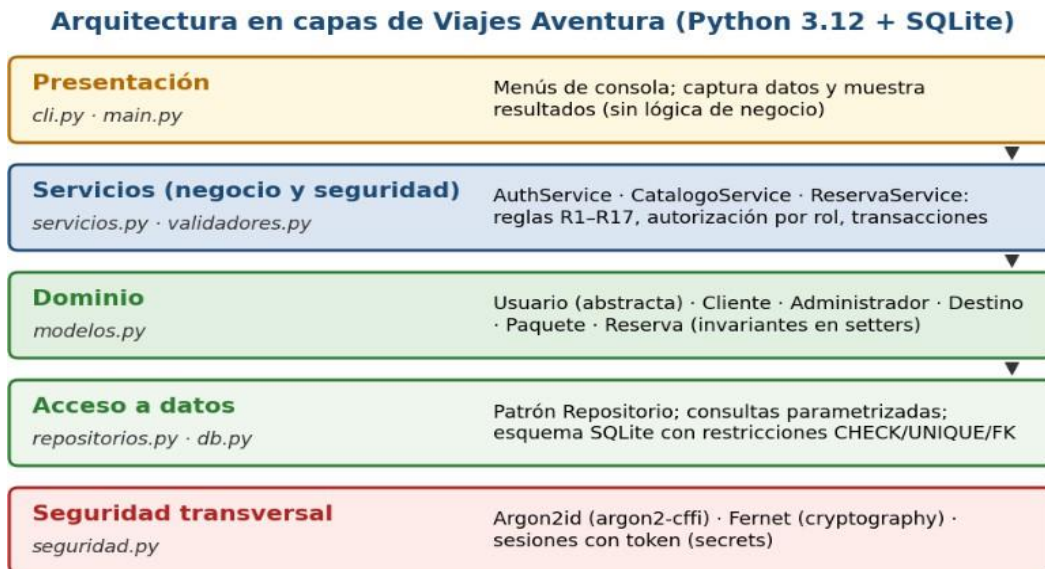


Figura 10. Arquitectura en capas.

3.6 Trazabilidad entre requerimientos y modelos (Indicador 4.1.2.I.8)

La matriz vincula cada requerimiento con los modelos que lo representan, el código que lo implementa y la prueba automatizada que lo verifica. No existe requerimiento sin modelo, ni modelo sin requerimiento.

Tabla 9. Matriz de trazabilidad entre requerimientos, modelos, código y pruebas.

Req.	Caso de uso	BPMN	Clases / métodos principales	Tablas	Prueba automatizada (tests/test_reglas.py)
RF-01 (R1, R2)	CU-07	BPMN 4	Destino (setters) · CatalogoService.registrar_destino · DestinoRepositorio.crear	Destinos	test_R1_nombre_usuario..., test_R2_costo_mayor_que_cero
RF-02 (R7)	CU-07	—	CatalogoService.modificar_destino · DestinoRepositorio.actualizar	Destinos	test_modificar_destino, test_R7_precio_con_gelado...
RF-03, RF-04 (R8)	CU-07	BPMN 4	CatalogoService.eliminar_destino · DestinoRepositorio.establecer_estado_en_paquetes / marcar_no_disponible	destinos, paquete_destinos	test_R8_eliminar_si_libre_y_no_disponible_si_en_paquete

Req.	Caso de uso	BPMN	Clases / métodos principales	Tablas	Prueba automatizada (tests/test_reglas.py)
RF-05 (R3, R4, R5)	CU-08	BPMN 3	Paquete (validaciones) · CatalogoService.crear_paquete · PaqueteRepositorio.crear	paquetes, paquete_destinos	test_R3_entre_2_y_5_destinos..., test_R5_fechas_y_cupo
RF-06 (R6)	CU-09	BPMN 3	Paquete.calcular_precio · Paquete.validar_margen	Paquetes	test_R6_precio_suma_mas_margen, test_R6_margen_negativo..., test_calculo_sin_publicar
RF-07 (R7)	CU-08	BPMN 3	Paquete.precio_por_persona (congelado) · PaqueteRepositorio.obtener	paquetes, paquete_destinos	test_R7_precio_congelado_tras_cambiar_costo
RF-08 (R14, R15)	CU-03	BPMN 5	CatalogoService.listar_paquetes · Paquete.esta_vigente · PaqueteRepositorio.personas_reservadas	paquetes, reservas	test_R14_no_supera_cupo, test_R15_paquete_vencido
RF-09 (R9, R10)	CU-01	BPMN 1	Cliente · AuthService.registrar_cliente · ServicioPassword.hashear · Cifrador	Usuarios	test_R9_correounico..., test_R10_password_no_en_texto_plano, test_politica_de_contrasenas
RF-10 (R11)	CU-02	BPMN 2	AuthService.login · GestorSesiones · ServicioPassword.verificar	Usuarios	test_login_mensaje_generico..., test_bloqueo_tras_5_intentos..., test_sesion_expira
RF-11 (R12–R16)	CU-04	BPMN 5	ReservaService.reservar · Reserva.calcular_total · ReservaRepositorio.crear	reservas	test_R13_total_y_R12_datos, test_R14..., test_R15..., test_R16_al_menos_una_persona
RF-12 (R11)	CU-05	—	ReservaService.mis_reservas · ReservaRepositorio.listar_por_usuario	reservas	test_R11_cliente_solo_ve_y_cancela_las_suyas
RF-13 (R11)	CU-02	BPMN 2	Usuario.permisos / puede · AuthService.autorizar	usuarios	test_R11_requiere_autenticacion_y_rol

Req.	Caso de uso	BPMN	Clases / métodos principales	Tablas	Prueba automatizada (tests/test_reglas.py)
RF-14	Todos	Todos	módulo validadores · setters de modelos	CHECK de todas las tablas	test_validadores, test_inyeccion_sql
RF-15 (R17)	CU-05	BPMN 1	Cliente.rut_enmascarado · Cifrador	usuarios	test_R17_enmascarado..., test_R17_rut_y_telefono_cifrados_en_bd
RF-16	CU-06	—	ReservaService.cancelar · ReservaRepositorio.cancelar	reservas	test_sin_duplicados_activos_y_cancelacion_libera_cupo
RF-17	CU-10	—	CatalogoService.listar_paquetes(incluir_vencidos=True)	paquetes	Verificado en ejecución manual (menú administrador)
RF-18	CU-04	BPMN 5	Índice único parcial ux_reserva_activa · ReservaRepositorio.crear	reservas	test_sin_duplicados_activos_y_cancelacion_libera_cupo

Validación de coherencia

Se comprobó que: (1) los 10 casos de uso cubren los 18 requerimientos funcionales; (2) cada decisión (compuerta) de los BPMN corresponde a una regla R# y a un método del código; (3) todas las clases del diagrama UML existen en el código con los mismos nombres, atributos y relaciones; (4) cada tabla del modelo de datos tiene una restricción por cada regla de integridad que le aplica; y (5) cada requerimiento funcional “Debe” tiene al menos una prueba automatizada aprobada (matriz anterior) y cada requerimiento no funcional tiene un método de verificación, por prueba o por inspección, indicado en la Tabla 10.

Tabla 10. Verificación de los requerimientos no funcionales.

RNF	Método de verificación	Evidencia
RNF-01	Prueba automatizada e inspección de la base de datos	test_R10_password_no_en_texto_plano
RNF-02	Prueba automatizada	test_R17_rut_y_telefono_cifrados_en_bd
RNF-03	Prueba automatizada con cargas de ataque	test_inyeccion_sql
RNF-04	Prueba automatizada	test_bloqueo_tras_5_intentos..., test_login_mensaje_generico...
RNF-05	Inspección del esquema (CHECK, UNIQUE, FOREIGN KEY) y prueba automatizada	test_R14_no_supera_cupo, test_sin_duplicados_activos...

RNF	Método de verificación	Evidencia
RNF-06	Medición con la base de demostración: cronometrar con time.perf_counter la reserva y el listado de paquetes	Listado de paquetes: 0.0028 s (2.8 ms) Reserva de paquete: 0.0065 s (6.5 ms) Tiempo total de operación: 0.0093 s (9.3 ms)
RNF-07	Inspección del menú de consola y de los mensajes de error	Ejecución del flujo de demostración (Anexo B)
RNF-08	Prueba automatizada e inspección de la estructura de módulos	Las 27 pruebas de tests/test_reglas.py; módulos del Anexo C
RNF-09	Inspección de las dependencias y del motor de base de datos	requirements.txt; SQLite del módulo estándar sqlite3
RNF-10	Inspección del registro tras ejecutar el flujo de demostración	Búsqueda de RUT, teléfono y contraseña en data/app.log
RNF-11	Prueba automatizada	test_R6_precio_suma_mas_margen, test_R13_total_y_R12_datos

4. Planificación con metodologías ágiles (criterio 4.1.3)

4.1 Marco de trabajo y roles

Se eligió Scrum porque el equipo es pequeño (2 a 3 personas), el plazo es corto (4 semanas con un entregable final) y el caso reconoce vacíos que se descubren y resuelven de forma iterativa. Cada sprint dura una semana y termina con un incremento revisable, lo que reduce el riesgo de llegar a la semana 18 con un sistema incompleto.

Tabla 11. Roles y responsabilidades del equipo.

Rol Scrum	Responsable	Responsabilidades
Product Owner (representante del cliente)	Sergio Ruiz	Mantiene y prioriza el Product Backlog según el valor para Viajes Aventura; aclara las reglas del caso y aprueba los supuestos S1–S10; acepta o rechaza cada historia al cierre del sprint.
Scrum Master y Líder de Proyecto	Benjamín Comanato	Coordina actividades, dirige las reuniones de seguimiento, elimina impedimentos, supervisa el cumplimiento de tareas y mantiene la comunicación con el docente (responsabilidades del Líder indicadas en la guía).
Equipo de desarrollo: dominio, datos y servicios	Benjamín Comanato y Sergio Ruiz (compartido)	Modelos de dominio, repositorios, servicios de catálogo y reservas, pruebas de reglas de negocio.
Equipo de desarrollo: seguridad y persistencia	Sergio Ruiz y Bernardo Fuentes (compartido)	Esquema SQLite, autenticación Argon2id, cifrado Fernet, sesiones, validadores y pruebas de seguridad.
Equipo de desarrollo: análisis, modelado y calidad	Bernardo Fuentes	Requerimientos, casos de uso, BPMN, UML, interfaz de consola, manual y revisión de calidad (QA) del informe.

Criterio de asignación. El Líder de Proyecto, Benjamín Comanato, actúa también como Scrum Master porque la guía le asigna coordinar actividades, dirigir las reuniones de seguimiento y comunicarse con el docente; además comparte con Sergio Ruiz el dominio y los servicios, por ser el núcleo que más depende del modelo UML. El Product Owner recae en Sergio Ruiz, quien al participar en el dominio, la seguridad y la persistencia prioriza el backlog con conocimiento directo de los requisitos de mayor riesgo del caso. La seguridad y la persistencia (esquema SQLite, Argon2id y Fernet) se comparten entre Sergio Ruiz y Bernardo Fuentes, de modo que cada pieza crítica tenga dos integrantes capaces de explicarla. Bernardo Fuentes tiene a su cargo el análisis, el modelado y la revisión de calidad (QA) del informe. Así, cada rol tiene responsables asignados, las responsabilidades compartidas están identificadas en la tabla y cada integrante puede fundamentar su aporte en la defensa.

Ceremonias y acuerdos

- **Sprint Planning** (inicio de cada semana, 45 min): se elige el alcance del sprint y se estima en puntos de historia (Fibonacci).
- **Reunión diaria breve** (15 min, presencial o por mensajería): qué hice, qué haré, qué me bloquea.
- **Sprint Review y Retrospectiva** (cierre semanal, 45 min): demostración del incremento y una mejora concreta para el siguiente sprint.
- **Definición de Hecho (DoD):** código revisado por otro integrante, pruebas automatizadas aprobadas, reglas R# trazadas, sin datos sensibles en logs, documentación actualizada y cambios integrados en el repositorio compartido.

4.2 Product Backlog priorizado

Las historias de usuario derivan directamente de los requerimientos (columna Req.). Puntos = esfuerzo relativo; Prior. según MoSCoW. La historia HU-16 se divide en HU-16a, HU-16b y HU-16c para que las pruebas avancen junto con cada incremento; el backlog suma 100 puntos.

Tabla 12. Product Backlog priorizado.

ID	Historia de usuario	Req.	Prior.	Pts	Sprint
HU-01	Como analista, quiero requerimientos y supuestos documentados para construir lo que el negocio necesita.	RF-01...RF-18	M	5	1
HU-02	Como equipo, quiero modelos UML y BPMN para compartir una visión común y trazable.	Todos	M	8	1
HU-03	Como equipo, quiero un esquema de base de datos con restricciones para proteger la integridad.	RNF-05	M	5	1
HU-04	Como administrador, quiero registrar y listar destinos para mantener el catálogo.	RF-01, RF-04	M	5	2
HU-05	Como administrador, quiero modificar un destino sin alterar los paquetes ya publicados.	RF-02, RF-07	M	3	2

ID	Historia de usuario	Req.	Prior.	Pts	Sprint
HU-06	Como administrador, quiero eliminar o desactivar un destino según esté o no en paquetes.	RF-03	M	5	2
HU-07	Como administrador, quiero crear un paquete con 2 a 5 destinos, fechas, cupo y margen.	RF-05	M	8	2
HU-08	Como administrador, quiero que el sistema calcule el precio y lo congele al publicar.	RF-06, RF-07	M	5	2
HU-09	Como cliente, quiero registrarme con datos protegidos para reservar con seguridad.	RF-09, RNF-01, RNF-02	M	8	3
HU-10	Como usuario, quiero iniciar sesión con bloqueo ante intentos fallidos y permisos por rol.	RF-10, RF-13, RNF-04	M	8	3
HU-11	Como cliente, quiero consultar paquetes vigentes con cupo y precio.	RF-08	M	3	3
HU-12	Como cliente, quiero reservar un paquete sin superar el cupo ni reservar fechas vencidas.	RF-11, RF-18	M	8	3
HU-13	Como cliente, quiero consultar mi historial de reservas con mis datos sensibles ocultos.	RF-12, RF-15	M	3	3
HU-14	Como cliente, quiero cancelar una reserva para liberar el cupo si desisto.	RF-16	S	3	4
HU-15	Como administrador, quiero ver todos los paquetes, incluidos los vencidos.	RF-17	S	2	4
HU-16a	Como equipo, quiero validar las entradas del catálogo y probar las reglas de destinos y paquetes (R1–R8).	RF-14, RNF-08	M	3	2
HU-16b	Como equipo, quiero validar las entradas de registro y reserva y probar las reglas de autenticación, cupo y fechas (R9–R17).	RF-14, RNF-08	M	3	3
HU-16c	Como equipo, quiero completar la cobertura y ejecutar las 27 pruebas con evidencia de aprobación.	RF-14, RNF-08	M	2	4
HU-17	Como equipo, quiero evaluar la seguridad y revisar críticamente el código generado con IA.	RNF-03, RNF-10	M	5	4
HU-18	Como equipo, quiero el informe técnico y la preparación de la defensa individual.	—	M	8	4

4.3 Sprint Backlog por iteración

Tabla 13. Sprint Backlog por iteración.

Sprint	Objetivo	Tareas principales	Incremento entregable	Pts
1 · Semana 15 Análisis y diseño	Entender el problema y dejar el diseño aprobado	Analizar caso y guía; redactar RF/RNF y supuestos (HU-01). Casos de uso, 5 BPMN y diagramas de clases (HU-02). Modelo de datos (HU-03). Crear repositorio y Product Backlog.	Documento de requerimientos y modelos; esquema SQL; backlog	18
2 · Semana 16 Núcleo y persistencia	Catálogo funcionando con base de datos	Clases de dominio con validaciones; módulo de validación; repositorios SQLite; CRUD de destinos (HU-04 a HU-06); paquetes y cálculo de precio (HU- 07, HU-08); pruebas de las reglas de destinos y paquetes (HU-16a).	CRUD de destinos y paquetes con persistencia y pruebas de R1 a R8	29
3 · Semana 17 Seguridad y reservas	Acceso seguro y reservas completas	Registro con cifrado y hash (HU-09); login, bloqueo y roles (HU-10); consulta de paquetes (HU-11); reservas con cupo transaccional (HU-12); historial de reservas (HU-13); pruebas de autenticación y reservas (HU-16b); interfaz de consola.	Aplicación completa de punta a punta con autenticación	33
4 · Semana 18 Calidad y entrega	Cerrar, verificar y entregar	Cancelación de reservas (HU-14) y listado completo de paquetes para el administrador (HU-15); completar y ejecutar las 27 pruebas (HU-16c); pip-audit de dependencias; evaluación de seguridad y registro del uso de IA (HU-17); informe final, manual y ensayo de defensa (HU-18); carga en AAI antes de las 23:00 h.	Informe técnico, código fuente y aplicación funcional	20

Los puntos suman 100 (18 + 29 + 33 + 20). El Sprint 1 es más liviano por concentrarse en análisis y diseño, y el Sprint 4 por dedicarse a cerrar, verificar y entregar; la mayor carga técnica se reparte entre los sprints 2 y 3.

4.4 Tiempos, entregables y riesgos

Tabla 14. Cronograma, hitos y entregables.

Semana	Hito	Entregable	Responsable	Criterio de avance
15	Fin del Sprint 1	Requerimientos, modelos UML/BPMN y backlog	Bernardo Fuentes (con el Líder)	Todas las reglas R1–R17 trazadas a un requerimiento
16	Fin del Sprint 2	Catálogo con persistencia y pruebas	Benjamín Comanato	CRUD de destinos y paquetes sin errores; pruebas R1–R8 aprobadas
17	Fin del Sprint 3	Autenticación, reservas y consola	Sergio Ruiz (con el Líder)	Flujo registro → login → reserva → historial operativo
18	Entrega final	Informe Word + código fuente + BD funcional	Líder de Proyecto	Archivos cargados en AAI antes de las 23:00 h; no se acepta correo
18+	Defensa	Defensa argumentativa individual	Cada integrante	Explica aporte, decisiones, IA, seguridad y persistencia

Tabla 15. Riesgos del proyecto y su mitigación.

Riesgo del proyecto	Prob.	Impacto	Mitigación
Integración tardía del trabajo de los integrantes	Media	Alto	Repositorio compartido y integración al cierre de cada sprint
Código de IA no comprendido o con fallos de seguridad	Media	Alto	Revisión cruzada, pruebas automatizadas y registro de decisiones (capítulo 7)
Ambigüedad del caso	Alta	Medio	Supuestos explícitos S1–S10 validados con el Product Owner
Falla de carga en AAI de último minuto	Baja	Alto	Cargar con un día de anticipación (recomendación de la guía)

5. Implementación en Python con persistencia (criterio 4.1.4)

5.1 Tecnologías y organización del código

Tabla 16. Tecnologías utilizadas y su justificación.

Componente	Elección	Justificación
Lenguaje	Python 3.12 (compatible con 3.10+)	Lenguaje del curso; tipado con anotaciones, dataclasses y excepciones claras.
Persistencia	SQLite con el módulo estándar sqlite3	Sin servidor; adecuada a 3 socios y ~214 reservas por temporada; soporta transacciones, claves foráneas y restricciones CHECK.
Hash de contraseñas	argon2-cffi (PyPI)	Argon2id, algoritmo recomendado por OWASP; sal y parámetros incluidos en el hash.
Cifrado de datos	cryptography · Fernet (PyPI)	Cifrado simétrico autenticado: confidencialidad más detección de alteración.
Pruebas	unittest (biblioteca estándar)	Sin dependencias extra; 27 pruebas ejecutables con un comando.

5.2 Programación orientada a objetos aplicada

Tabla 17. Aplicación de los principios de programación orientada a objetos.

Principio	Dónde se aplica	Ejemplo concreto
Abstracción	Usuario (clase abstracta con ABC) y Repositorio (base abstracta)	Usuario declara permisos como propiedad abstracta: no se puede instanciar sin definir el rol.
Encapsulamiento	Atributos privados con propiedades que validan	Destino.costo_base rechaza valores ≤ 0 (R2); un objeto inválido no puede existir.
Herencia	Cliente y Administrador heredan de Usuario; los cuatro repositorios heredan de Repositorio	Cliente agrega RUT y teléfono; Administrador no los necesita.
Polimorfismo	Usuario.puede(permiso) opera igual para cualquier rol, cada uno con sus permisos	AuthService.autorizar(token, "gestionar_destinos") acepta a Administrador y rechaza a Cliente sin condicionales por tipo.
Composición y responsabilidad única	Paquete contiene Destinos; los servicios reciben repositorios por inyección	fabrica.construir() ensambla las dependencias; las pruebas usan una base temporal.

5.3 Persistencia de datos

El esquema (sección 3.4) se crea automáticamente al conectar. Las reglas se aplican en dos niveles: en el dominio (mensajes claros para el usuario) y en la base de datos (defensa en profundidad). Las operaciones que modifican varias tablas o dependen de un dato compartido, como el cupo, se ejecutan dentro de una transacción con BEGIN IMMEDIATE, que bloquea a otros escritores y evita la sobreventa que sufrió Matías.

5.4 Funcionalidades CRUD

Tabla 18. Matriz CRUD por entidad.

Entidad	Crear	Leer	Actualizar	Eliminar	Actor
Destino	registrar_destino	listar_destinos · obtener	modificar_destino	eliminar_destino (física si está libre; lógica "no disponible" si está en un paquete)	Administrador
Paquete	crear_paquete (publica y congela precio)	listar_paquetes · consultar_paquete · estimar_precio	No aplica: el precio publicado es inmutable (R7)	No aplica: se conserva como historial (S3)	Administrador (crear) · todos (leer)
Reserva	reservar	mis_reservas	cancelar (cambia estado; total inmodificable, R13)	No se elimina: se cancela (S4)	Cliente
Usuario	registrar_cliente · crear_administrador	login (verificación)	Reintento de hash automático si cambian los parámetros	No incluido en el alcance	Cliente / sistema

5.5 Pruebas y verificación

Se escribieron 27 pruebas automatizadas con unittest, cada una asociada a una regla R# o a un control de seguridad. Usan una base de datos temporal y relojes simulados, por lo que son repetibles. Resultado de la ejecución final: 27 pruebas, 0 fallos, 0 errores (OK).

Tabla 19. Pruebas automatizadas por grupo.

Grupo	Pruebas	Qué verifican
Destinos (4)	R1 nombre único · R2 costo > 0 · R8 eliminar o desactivar · modificar destino	Integridad del catálogo y baja lógica
Paquetes (6)	R6 precio y margen · R6 margen negativo · R3 entre 2 y 5 destinos · R5 fechas y cupo · R7 precio congelado · cálculo sin publicar	Cálculo automático y precio fijo
Reservas (7)	R13 total · R14 cupo · R15 paquete vencido · R16 mínimo una persona · sin duplicados y cancelación · R11 aislamiento entre clientes · R11 autenticación y rol	Las tres fallas centrales del caso: sobrecupo, fechas vencidas y duplicados
Seguridad (10)	R10 hash · R17 cifrado · R17 enmascarado · R9 correo único · login genérico · bloqueo de cuenta · inyección SQL · política de contraseñas · expiración de sesión · validadores	Autenticación, validación y protección de datos

Además, se realizó una prueba de humo de la interfaz de consola con datos de demostración (los 6 destinos y 3 paquetes del caso): registro de un cliente, inicio de sesión, consulta de paquetes (Norte Grande en 5 días a \$516.000 por persona; Sur profundo a \$990.000) y una reserva de 3 personas del paquete de fin de semana por \$298.800.

5.6 Limitaciones conocidas

- La interfaz es de consola; una interfaz web o de escritorio queda como trabajo futuro y solo requiere reemplazar la capa de presentación.
- SQLite atiende a un volumen bajo y a pocos usuarios simultáneos; si la agencia crece, el patrón Repositorio permite migrar a PostgreSQL cambiando solo la capa de acceso a datos.
- El pago y su verificación son manuales (fuera de alcance, supuesto S5).

6. Acceso seguro a la solución (criterio 4.1.5)

6.1 Librerías de repositorios oficiales y criterios de calidad

Para autenticación y protección de datos no se implementó criptografía propia: se usaron librerías publicadas en el repositorio oficial de Python (PyPI) y la biblioteca estándar. Cada recomendación de IA sobre seguridad se validó con los criterios técnicos de la Tabla 21 antes de aceptarse.

Tabla 20. Librerías de repositorios oficiales utilizadas.

Librería	Repositorio	Versión usada	Uso en el sistema
argon2-cffi	PyPI (pypi.org/project/argon2-cffi)	25.1.0	Hash de contraseñas con Argon2id y sal única (RNF-01); verificación y detección de necesidad de rehash.
cryptography (Fernet)	PyPI (pypi.org/project/cryptography)	46.0.6	Cifrado autenticado en reposo de RUT y teléfono (RNF-02).
secrets	Biblioteca estándar de Python	3.12	Tokens de sesión aleatorios criptográficamente seguros.
sqlite3	Biblioteca estándar de Python	3.12	Consultas parametrizadas (RNF-03).

Tabla 21. Criterios técnicos de calidad para validar librerías y recomendaciones de IA.

Criterio	Cómo se verificó	Resultado
Origen oficial	El paquete existe en PyPI con el nombre exacto y mantenedores reconocidos (previene paquetes falsos sugeridos por IA).	Cumple
Estándar reconocido	El algoritmo aparece en guías de referencia (OWASP Password Storage Cheat Sheet recomienda Argon2id; Fernet es la receta de alto nivel de cryptography).	Cumple
Mantenimiento activo	Versiones recientes publicadas y documentación oficial vigente.	Cumple
Sin vulnerabilidades conocidas	Ejecución de pip-audit sobre requirements.txt: "No known vulnerabilities found".	Cumple
API difícil de usar mal	Parámetros seguros por defecto (Argon2id, sal automática, autenticación del mensaje en Fernet).	Cumple
Comprensión del equipo	Cada integrante puede explicar qué hace cada llamada (requisito de la guía).	Cumple

6.2 Mecanismo de autenticación

El flujo (BPMN 2) es: validar formato del correo → consultar el usuario con SQL parametrizado → revisar bloqueo → verificar el hash Argon2id → registrar fallo o éxito → abrir una sesión con un token aleatorio de 256 bits (secrets.token_urlsafe) que expira tras 30 minutos de inactividad. Toda operación posterior exige el token y el permiso del rol (autorización).

6.3 Validación rigurosa de credenciales (Indicador 4.1.5.G.18)

Tabla 22. Validación de credenciales por escenario.

Escenario	Comportamiento del sistema	Prueba
Correo con formato inválido o con código SQL	Se rechaza con el mensaje genérico; no se consulta la base de datos con ese texto	test_inyeccion_sql, test_login_mensaje_generico...
Correo inexistente	Mismo mensaje y mismo costo de tiempo que una clave incorrecta (hash señuelo): no permite descubrir qué correos están registrados	test_login_mensaje_generico...
Contraseña incorrecta	Mensaje genérico y conteo del intento fallido	test_login_mensaje_generico...
5 intentos fallidos	Bloqueo de 15 minutos aun con la clave correcta; se libera al pasar el tiempo	test_bloqueo_tras_5_intentos...
Contraseña débil al registrarse	Se exigen 10 a 128 caracteres con mayúsculas, minúsculas y números	test_politica_de_contrasenas
Token falso o sesión expirada	Se exige iniciar sesión nuevamente	test_R11_requiere_autenticacion_y_rol, test_sesion_expira
Cliente intenta operación de administrador	Rechazo de autorización	test_R11_requiere_autenticacion_y_rol
Cliente intenta ver o cancelar reserva ajena	Responde “no existe” sin confirmar que pertenece a otro cliente	test_R11_cliente_solo_ve_y_cancela_las_suyas

7. Protección de la información sensible (Indicador 4.1.5.I.19)

Tabla 23. Protección de la información sensible.

Dato	Mecanismo	Dónde y cómo
Contraseña	Hash Argon2id con sal única (no reversible)	usuarios.password_hash; nunca se guarda, registra ni muestra la clave original (R10). Al iniciar sesión se reevalúa si el hash requiere actualización.
RUT y teléfono	Cifrado autenticado Fernet (AES-128- CBC + HMAC-SHA256)	usuarios.rut_cifrado y telefono_cifrado (BLOB); se descifran solo en memoria cuando el propio cliente inicia sesión.
Visualización	Enmascarado	rut_enmascarado y telefono_enmascarado muestran solo los últimos caracteres; ningún listado ni mensaje de error incluye el dato completo (R17).
Clave de cifrado	Variable de entorno VIAJES_FERNET_KEY o archivo con permisos 0600	La clave no está en el código fuente ni en la base de datos; si se pierde, los datos cifrados no pueden recuperarse (se documenta en el manual).
Registros (log)	Sin datos sensibles	app.log solo guarda identificadores numéricos y eventos (“login fallido”); nunca RUT, teléfono ni contraseñas.
Errores	Mensajes controlados	Las excepciones de dominio tienen textos seguros; los errores inesperados se registran y al usuario solo se le informa un mensaje genérico.

7.1 Análisis de amenazas y controles

Tabla 24. Amenazas, controles y riesgo residual.

Amenaza (categoría)	Control implementado	Riesgo residual
Fuerza bruta o adivinación de claves (Spoofing)	Argon2id (costoso de atacar), bloqueo temporal, política de contraseñas	Bajo; no hay segundo factor (MFA)
Enumeración de usuarios	Mensajes genéricos y hash señuelo para igualar tiempos	Bajo
Inyección SQL (Tampering)	Consultas parametrizadas, validación de entrada y restricciones en la BD	Bajo
Robo del archivo de base de datos (Information Disclosure)	Contraseñas con hash y RUT/teléfono cifrados	Medio: si la clave Fernet está en el mismo equipo, puede obtenerse; usar variable de entorno y permisos del sistema
Acceso a datos ajenos (Elevation of Privilege)	Autorización por rol y filtro por usuario_id en todas las consultas de reservas	Bajo
Sobreventa o reserva vencida (Tampering / negocio)	Transacción BEGIN IMMEDIATE y validación de fecha y cupo	Bajo
Fuga por mensajes o logs (Information Disclosure)	Mensajes seguros y log sin datos sensibles	Bajo
Negación de responsabilidad (Repudiation)	Log de eventos de seguridad y reservas con fecha y cliente	Medio: el log es un archivo local sin firma
Dependencias vulnerables (cadena de suministro)	Solo PyPI oficial, versiones fijadas como mínimo y pip- audit sin hallazgos	Bajo; repetir pip-audit periódicamente

7.2 Evaluación y optimización de la seguridad con apoyo de IA (Indicador 4.1.5.I.20)

La evaluación se hizo revisando el código contra la lista de amenazas anterior y pidiendo a la herramienta de IA una revisión adversaria. Cada hallazgo se contrastó con la documentación oficial y con una prueba automatizada antes de aplicar la mejora:

Tabla 25. Hallazgos de seguridad y mejoras aplicadas.

Hallazgo	Mejora aplicada	Verificación
Responder distinto para “correo no existe” y “clave incorrecta” permite descubrir cuentas	Mensaje único y hash señuelo para igualar tiempos	test_login_mensaje_generico...
Sin límite de intentos, un atacante puede probar claves indefinidamente	Bloqueo de 15 minutos tras 5 fallos	test_bloqueo_tras_5_intentos...

Hallazgo	Mejora aplicada	Verificación
La clave de cifrado podría quedar legible por otros usuarios del equipo	Archivo creado con permisos 0600 y prioridad a la variable de entorno	Verificación de permisos en ejecución (600)
El validador de enteros aceptaba "1.5" como 15 al eliminar los puntos (error de la primera versión)	Se exige solo dígitos con una expresión regular estricta	test_R16_al_menos_u na_persona (la prueba falló antes del arreglo)
Conteo de cupo y guardado separados permitían sobreventa concurrente	Transacción BEGIN IMMEDIATE	test_R14_no_supera_c upo
Duplicados por reintentos o dos socios	Índice único parcial en la base de datos	test_sin_duplicados_a ctivos...
Posibles dependencias con vulnerabilidades	Auditoría con pip-audit	Sin vulnerabilidades conocidas

7.3 Riesgos residuales y recomendaciones

- Guardar la clave de cifrado en un gestor de secretos o variable de entorno del servidor, separada de la base de datos; definir un procedimiento de respaldo y rotación (por ejemplo, MultiFernet).
- Agregar segundo factor de autenticación y recuperación de contraseña cuando exista envío de correos (hoy fuera de alcance).
- Si se emigra a una interfaz web, usar HTTPS, cookies seguras y protección CSRF; y si se pasa a PostgreSQL, usar un usuario con mínimos privilegios.
- Repetir pip-audit y actualizar dependencias en cada entrega.

8. Uso crítico de herramientas de inteligencia artificial

La guía permite y espera el uso de IA como apoyo en la codificación, con tres condiciones: el equipo comprende el código, lo valida técnicamente y justifica las decisiones de uso, modificación o descarte. Por eso la IA se trató como un asistente que propone y al equipo le correspondió decidir. Ningún fragmento se incorporó sin pruebas y sin que al menos un integrante pudiera explicarlo.

8.1 Proceso de revisión aplicado

1. **Proponer:** la IA sugiere una alternativa (algoritmo, estructura, consulta).
2. **Verificar en fuente oficial:** documentación de Python, PyPI, OWASP; se confirma que la librería y sus parámetros existen.
3. **Probar:** se escribe o ejecuta una prueba automatizada que demuestre el comportamiento esperado.
4. **Decidir y registrar:** aceptar, modificar o descartar, anotando el motivo (tabla siguiente).

8.2 Registro de contribuciones, análisis y decisiones

Tabla 26. Registro de contribuciones de IA, análisis y decisiones.

N.º	Tema	Propuesta, alternativa o resultado de la IA	Análisis y validación del equipo	Decisión	Justificación
1	Hash de contraseñas	Alternativas consideradas: bcrypt o Argon2.	Se contrastó con OWASP: Argon2id es la primera opción; bcrypt tiene límite de 72 bytes en la clave.	Modificada: Argon2id	Mayor resistencia a hardware especializado; sal y parámetros incluidos en el hash.
2	Persistencia	Alternativa considerada: usar un ORM completo.	El dominio tiene 5 tablas y reglas simples; un ORM agrega dependencias y oculta el SQL que se debe entender y defender.	Descartada: sqlite3 estándar	Menor superficie de ataque, control explícito de transacciones y consultas parametrizadas.
3	Precios	Alternativa habitual en ejemplos: calcular con números flotantes (precio * 1.2).	La coma flotante puede introducir errores de redondeo en dinero; se comprobó con la aritmética exacta.	Modificada: Decimal y enteros CLP	Precisión exacta en precios y totales (RNF-11).
4	Validación de enteros	El primer validador eliminaba puntos para aceptar "1.000".	La prueba de cantidad de personas detectó que "1.5" se interpretaba como 15.	Modificada: solo dígitos	Un error de validación podía alterar un total cobrado; se corrigió y quedó cubierto por prueba.
5	Datos de prueba	Entregó RUT de ejemplo con dígito verificador incorrecto.	El validador (módulo 11) los rechazó; se recalcularon a mano con el algoritmo.	Modificada	Las pruebas deben usar datos válidos para comprobar el validador.
6	Mensajes de login	Alternativa habitual en ejemplos: mensajes específicos ("correo no registrado").	Facilitan la enumeración de cuentas (OWASP Authentication Cheat Sheet).	Descartada	Mensaje genérico y hash sefueo para igualar tiempos.
7	Control de cupo	Alternativa simple: consultar el cupo y luego insertar.	Con dos reservas simultáneas ambas podrían pasar la validación.	Modificada: transacción BEGIN IMMEDIATE	Evita el sobrecupo que el caso reporta (6 reservas).
8	Clave de cifrado	Alternativa habitual en ejemplos: clave fija en el código.	Un secreto en el código fuente queda expuesto en el repositorio.	Descartada	Variable de entorno o archivo con permisos 0600.
9	Reservas duplicadas	Propuesta: índice único parcial (usuario, paquete) para reservas activas.	Se verificó en la documentación de SQLite y con prueba automatizada.	Aceptada	Protege contra duplicados aunque falle la capa de aplicación.
10	Interfaz	Alternativa considerada: aplicación web.	El alcance pide aplicación funcional con autenticación y persistencia; la web agrega riesgos (CSRF, sesiones) y tiempo.	Descartada para la v1	Consola ahora; la capa de servicios permite agregar web después.

8.3 Riesgos del uso de IA y cómo se controlaron

- **Código que “parece correcto” pero falla en casos límite:** se cubrió con 27 pruebas, incluyendo entradas maliciosas.
- **Librerías o funciones inexistentes:** se verificó cada dependencia en PyPI y su documentación oficial antes de agregarla.
- **Prácticas inseguras de ejemplo** (claves fijas, mensajes reveladores): se revisaron contra OWASP y se descartaron.
- **Dependencia excesiva:** cada integrante debe poder explicar y modificar los módulos que presenta en su defensa; el código final es el resultado de un refinamiento propio del equipo.

9. Conclusiones

La solución convierte las reglas y fallas del caso en un sistema verificable: el sobrecupo se evita con un cupo disponible calculado dentro de una transacción; las reservas vencidas se rechazan comparando con la fecha del día; el precio se calcula automáticamente y queda congelado, de modo que cambiar el costo de un destino no altera lo ya publicado ni lo ya cobrado; los destinos que dejan de operarse se desactivan sin romper los paquetes vendidos; y los datos sensibles dejan de circular en planillas para quedar cifrados y restringidos por rol.

Desde lo metodológico, el trabajo mostró que separar regla de negocio y requerimiento ordena el análisis, y que la trazabilidad (requerimiento → caso de uso → BPMN → clase → tabla → prueba) permite demostrar, y no solo afirmar, que el sistema responde al problema del cliente. Desde lo técnico, reforzar las reglas también en la base de datos y usar librerías oficiales de seguridad redujo el riesgo sin aumentar la complejidad.

Trabajo futuro: interfaz web con HTTPS, segundo factor de autenticación, recuperación de contraseña y notificaciones cuando exista envío de correos, informes de gestión para los socios, verificación automática de pagos y migración a un motor de base de datos cliente-servidor.

Referencias

Cuevas Álvarez, A. (2016). *Python 3*. RA-MA Editorial.

Jiménez de Parga, C. (2021). *UML: arquitectura de aplicaciones en Java, C++ y Python (1.ª ed.)*. Ra-Ma.

Sánchez Palacio, A. (2025). *ChatGPT y OpenAI: desarrollo y uso de herramientas de inteligencia artificial generativa*. RA-MA Editorial.

Object Management Group. (2011). *Business Process Model and Notation (BPMN)*, versión 2.0. <https://www.omg.org/spec/BPMN/2.0/>

OWASP Foundation. (s. f.). *Password Storage Cheat Sheet y Authentication Cheat Sheet*. <https://cheatsheetseries.owasp.org/>

Python Software Foundation. (s. f.). *sqlite3: DB-API 2.0 interface for SQLite databases y secrets: generate secure random numbers*. <https://docs.python.org/3/>

argon2-cffi. (s. f.). *Documentación oficial y repositorio del paquete en PyPI*.

<https://argon2-cffi.readthedocs.io/>

Python Cryptographic Authority. (s. f.). *cryptography: Fernet (symmetric encryption)*. <https://cryptography.io/>

Schwaber, K. y Sutherland, J. (2020). *La Guía de Scrum*. <https://scrumguides.org/>